Practical - 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition : Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

A. Theory

1. Do PHP endpoints return valid JSON responses?

Yes. api.php starts with header("Content-Type: application/json") and every response uses json_encode() with a consistent format:

```
{ "success": true/false, "message": "...", "data": [...] }
```

2. Are GET and POST requests handled properly?

Yes — all 4 HTTP methods are handled inside api.php using $_SERVER['REQUEST_METHOD']:

- GET → Returns all events as JSON array
- POST → Reads JSON body with file_get_contents("php://input") and inserts into DB
- PUT → Reads JSON body and updates the matching event
- DELETE → Reads JSON body and deletes the matching event

3. Are loading, success, and error states shown on the frontend?

Yes — three UI states are handled in app.js:

- Loading: The #loading div shows ⏳ Loading... while any Fetch request is in progress
- Success: Green text message appears for 3 seconds (auto-hides)
- Error: Red text message appears for 3 seconds (auto-hides)

4. Is input validation performed before database operations?

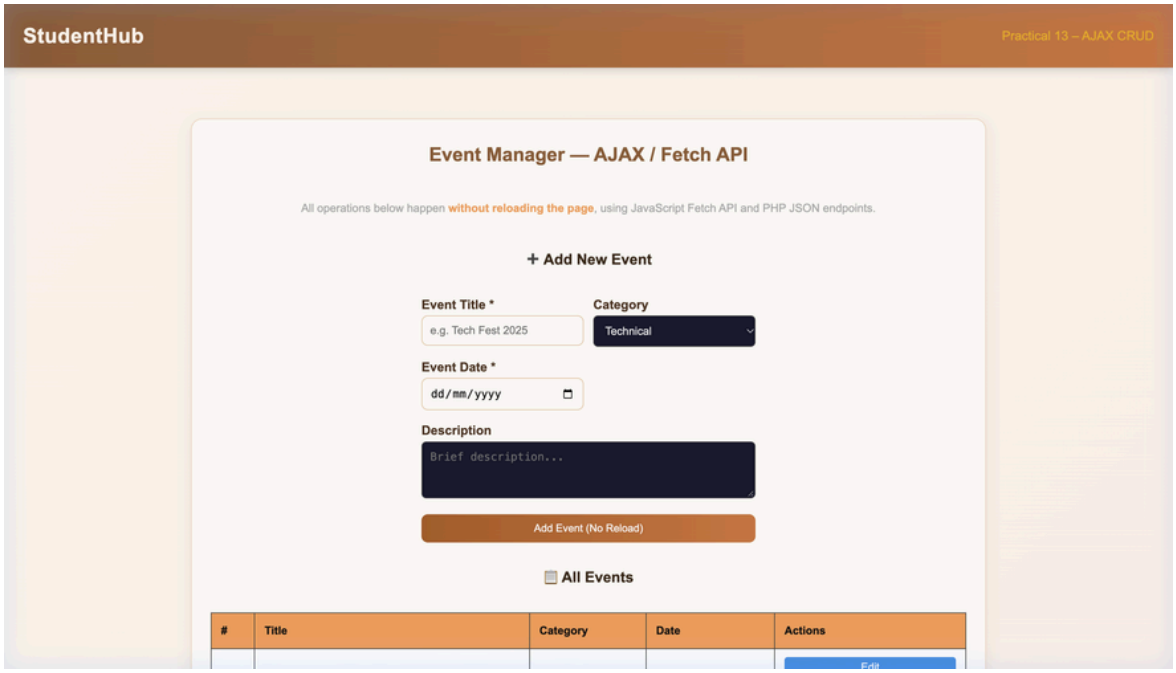
Yes — validation happens on both sides:

- Frontend (JS): Checks if Title and Date are empty before even sending the Fetch request
- Backend (PHP): Also checks for empty fields after receiving the JSON, returns an error JSON response if validation fails — so the API is safe even if someone bypasses the frontend

B. Implementation

api.php	PHP JSON API — handles GET, POST, PUT, DELETE requests
index.html	Frontend page — Add form, Events table, Edit form
app.js	JavaScript — all Fetch API calls, table rendering, messages
db_connect.php	Reused — basic MySQLi connection
CSS/script.css	Reused styles from previous practicals

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%213

E. Challenges Faced:

1. Reading JSON request body in PHP — Unlike regular form POSTs where data comes in $_POST, a Fetch API request sends JSON. You must read it using file_get_contents("php://input") and then json_decode(). This is a key difference for beginners to understand.
2. PUT and DELETE in PHP — PHP doesn't natively support $_PUT or $_DELETE superglobals, so both must use file_get_contents("php://input") — same as POST.
3. Preventing Page Reload — The form's onsubmit handler must call event.preventDefault() first. Without this, the browser reloads the page on every form submit, defeating the purpose of AJAX.
4. Updating the table without reload — After every successful add/edit/delete, loadEvents() is called again which re-fetches all events from the API and re-renders the table — giving a seamless real-time feel.

F. Learning Outcomes:

1. Learned what a JSON API is — a PHP file that returns structured JSON data instead of HTML.
2. Understood the Fetch API in JavaScript — how to make GET, POST, PUT, DELETE requests from the browser.
3. Learned how to read JSON request bodies in PHP using file_get_contents("php://input").
4. Understood async/await — using async function and await to handle asynchronous fetch calls without complex callback chains.
5. Learned how to do two-layer validation — frontend (JavaScript) and backend (PHP) for the same data.

Faculty Remarks: